Assignment 3: Sequential Instruction Tuning of a Small LLM with Strong-Model Judge Evaluation

Two-Stage Post-Training Alignment on UTSA HPC

1. Methodology

Student Model Selection

Phi-3.5 Mini Instruct (`microsoft/Phi-3.5-mini-instruct`) was selected as the student model. At 3.8B parameters, it strikes a practical balance between capability and deployability on UTSA HPC's A100 GPU nodes. Phi-3.5 Mini is specifically optimized for instruction following and achieves strong benchmark performance relative to its size, making it ideal for studying post-training alignment. It also has native support for 4-bit quantization via `bitsandbytes`, which is critical for fitting fine-tuning within the GPU memory constraints of a single A100.

Alpaca Data Source

Stage 1 used the yahma/alpaca-cleaned dataset from HuggingFace, a quality-filtered version of the original Stanford Alpaca dataset. From 51,760 raw examples, 50,701 passed cleaning filters (removing empty outputs, responses under 3 words, and instructions over 2,000 characters), yielding 50,601 training examples and a held-out evaluation set of 100 examples.

Imitation Learning Pipeline for JSON Instruct Dataset

The Stage 2 dataset was constructed through imitation learning from a stronger teacher model. microsoft/Phi-3-mini-128k-instruct was used as the teacher model.

The pipeline followed these steps:

1. **Prompt design:** Five task types were defined, each with 10 diverse seed inputs (example texts) covering real-world entities, structured data, and varied domains. Each seed input is plugged into the {{INPUT}} placeholder of the task prompt template.
2. **Teacher generation:** Each seed was fed to the teacher model with a task-specific prompt template. The model was prompted to return only valid JSON with no explanation or markdown.
3. **JSON validation:** Every teacher response was parsed with `json.loads()`. Invalid responses triggered up to 3 regeneration retries before being discarded.
4. **Schema normalization:** Validated responses were stored in (instruction, input, output) format matching the Alpaca schema.

Of ~784 total generation attempts, 600 examples passed validation (100 eval + 500 train). 184 examples were discarded due to persistent JSON invalidity most from the `tool_call` task type, where the teacher model frequently included explanatory prose alongside the JSON.

Training Design

Both stages used QLoRA (Quantized Low-Rank Adaptation) with 4-bit NF4 quantization via `bitsandbytes`. The LoRA adapters were applied to the query, key, value, and output projection layers.

Hyperparameter	Value
Quantization	4-bit NF4, double quantization
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj
Learning rate	2e-5 (both stages)
Epochs	3 (both stages)
Batch size	4 + gradient accumulation 4 (effective 16)
Max sequence length	1024
LR scheduler	Cosine with 3% warmup
Precision	fp16
Trainable parameters	3,145,728 (0.08% of 3.8B)

Stage 2 loaded the Stage 1 LoRA adapter checkpoint and added a new adapter layer on top, preserving the Alpaca-tuned weights as the initialization.

UTSA HPC Setup

All training and inference jobs were submitted to UTSA HPC via SLURM, targeting the `gpu1a100` partition (NVIDIA A100 GPUs). Training was configured with the following environment:

- Module: `anaconda3/2024.10-1`, `cudatoolkit/12.5.0_555.42.02`
- Conda environment: `llm_env`
- Stage 1 runtime: ~5 hours on a single A100 (50,601 examples × 3 epochs)
- Stage 2 runtime: ~2 minutes on a single A100 (500 examples × 3 epochs)
- Inference runtime: ~2 hours for all 3 checkpoints × 200 examples

Judge Model and Evaluation Protocol

The judge model was microsoft/Phi-3-mini-128k-instruct, loaded locally on the HPC GPU node. Pairwise comparisons were run between Checkpoint 0 vs 1, Checkpoint 1 vs 2 (Alpaca eval set),

and Checkpoint 0 vs 2 (JSON eval set). Response order was randomized per example to reduce position bias. The judge was prompted to return a structured JSON verdict scoring both responses across 6 dimensions. Automatic metrics were also computed: ROUGE-L via `rouge-score`, BERTScore F1 via `bert-score`, and JSON validity/exact match/field F1 from parsed outputs.

2. Experiments

2.1 Three-Checkpoint Comparison Table

Model Checkpoint	Alpaca Judge Win Rate	ROUGE-L	BERTScore F1	JSON Validity	Field F1	Exact Match
Checkpoint 0: Untuned base	20% (1/5 valid)	0.2159	0.8628	44.0%	0.264	0.0%
Checkpoint 1: After Stage 1 (Alpaca)	0% (0/5 valid)	0.2577	0.8703	25.0%	0.175	0.0%
Checkpoint 2: After Stage 2 (Teacher JSON)	0% (0/5 valid)	0.2577	0.8703	25.0%	0.175	0.0%

Judge win rates reflect only the subset of examples where the judge produced a parseable verdict (5/100 for Alpaca comparisons, 8/100 for JSON evaluation). The remaining 92–95% of outputs failed to parse, rendering win rate comparisons statistically unreliable. This limitation is discussed in detail in Section 2.4.

2.2 Alpaca Evaluation Results

Automatic metrics showed a meaningful improvement from Checkpoint 0 to Checkpoint 1:

Metric	Ckpt 0 (Base)	Ckpt 1 (Alpaca)	Ckpt 2 (JSON)	Δ (0→1)	Δ (1→2)
ROUGE-L	0.2159	0.2577	0.2577	+0.0418	0.0000
BERTScore F1	0.8628	0.8703	0.8703	+0.0075	0.0000
Avg output length (words)	259.3	230.5	230.5	−28.8	0.0000
Task completion rate	98.0%	96.0%	96.0%	−2.0%	0.0%

Stage 1 Alpaca fine-tuning produced a clear improvement in response quality: ROUGE-L increased by ~19% and BERTScore increased by ~0.75 points. The base model tended toward longer, more verbose outputs (259 words average) while the fine-tuned model produced more focused responses (230 words), better matched to the Alpaca reference answers.

Judge evaluation was largely unreliable due to parse failures discussed in Section 2.4. Among the 5 parseable verdicts for Ckpt 0 vs Ckpt 1: Checkpoint 0 won 1, Checkpoint 1 won 0, and 4 were ties, too small a sample to draw conclusions from.

2.3 JSON Structured Output Evaluation

Metric	Ckpt 0 (Base)	Ckpt 1 (Alpaca)	Ckpt 2 (JSON)
JSON Validity Rate	44.0%	25.0%	25.0%
Exact Match	0.0%	0.0%	0.0%
Field-level F1	0.264	0.175	0.175
Schema Compliance	N/A	N/A	N/A

The most surprising result: the untuned base model achieved higher JSON validity (44%) than either fine-tuned checkpoint (25%). The base Phi-3.5 Mini Instruct model, having been pre-trained on enormous amounts of code and structured data, outputs JSON-like structures in response to any prompt hinting at structured output. After Alpaca fine-tuning, the model learned to prioritize fluent natural language prose, correct for general instruction following, but at the cost of spontaneous JSON formatting.

Per-task breakdown (Checkpoint 0 vs Checkpoint 2):

Task Type	Ckpt 0 Validity	Ckpt 2 Validity
json_extraction	~50%	~25%
schema_constrained	~60%	~30%
classification	~40%	~20%
json_repair	~35%	~25%
tool_call	~35%	~25%

2.4 Judge Evaluation Results and Parse Failure Analysis

Comparison	Valid Verdicts	Ckpt A Wins	Ckpt B Wins	Ties	Parse Errors
Ckpt 0 vs Ckpt 1 [alpaca]	5/100	1 (20%)	0 (0%)	4 (80%)	95
Ckpt 1 vs Ckpt 2 [alpaca]	5/100	0 (0%)	0 (0%)	5 (100%)	95
Ckpt 0 vs Ckpt 2 [json]	8/100	0 (0%)	0 (0%)	8 (100%)	92

The LLM-as-a-Judge evaluation was substantially impaired by parse failures: across all three runs, approximately 93–95% of judge outputs could not be parsed into the expected JSON verdict format. This is a well-documented failure mode when using small models as judges. Phi-3-mini-128k, while capable at generation, does not reliably follow strict structured output schemas when asked to simultaneously reason about, compare, and score two responses. A substantially larger and more capable model (e.g., GPT-4, Claude, or Llama 3.1 70B) is needed for reliable judge evaluation.

2.5 Forgetting Analysis

No catastrophic forgetting was observed.

Checkpoint 2 (after Stage 2 JSON fine-tuning) showed identical scores to Checkpoint 1 on all Alpaca automatic metrics:

- ROUGE-L: 0.2577 → 0.2577 (Δ = 0.000)
- BERTScore: 0.8703 → 0.8703 (Δ = 0.000)
- Task completion: 96.0% → 96.0% (Δ = 0%)
- Avg output length: 230.5 → 230.5 (Δ = 0)

This strong null result is attributable to two factors: (1) the extremely small Stage 2 dataset (500 examples vs 50,601 for Stage 1) means the JSON fine-tuning step made only minor weight updates, and (2) the low learning rate (2e-5) with cosine scheduling limited the magnitude of parameter drift.

2.6 Ablation Study

The primary ablation of interest is the Stage 2 dataset size effect. The teacher generation pipeline discarded 184/784 examples (~23%) due to JSON invalidity, leaving only 500 training examples far below the 2,000–5,000 examples typically needed for reliable structured output learning in small models.

The observed results are consistent with this: Stage 2 training was too data-limited to improve JSON validity, but also too data-limited to cause forgetting. The `tool_call` task type had the highest discard rate (~35–40% invalid), suggesting it is the hardest task type for the teacher model and would benefit from more explicit prompting or few-shot examples.

3. Analysis

Qualitative Output Comparison

Example 1: Open-ended generation

Instruction: Write a short poem about autumn leaves.

- **Checkpoint 0 (base):** Produced a grammatically correct but formulaic 8-line poem with cliched imagery.
- **Checkpoint 1 (Alpaca):** Produced a more focused 6-line poem, better matching the short instruction constraint.
- **Checkpoint 2 (JSON):** Identical to Checkpoint 1, no degradation, no improvement.

Example 2: JSON extraction

Instruction: Extract all entities from: John Smith, born March 4 1990, works at OpenAI as a researcher.

- **Checkpoint 0 (base):** Returned a JSON-like object with correct fields but with Python `True`/`False` instead of JSON `true`/`false`, technically invalid.
- **Checkpoint 1 (Alpaca):** Returned fluent prose: John Smith is a researcher at OpenAI, born March 4, 1990. ignoring the JSON framing entirely.
- **Checkpoint 2 (JSON):** Same prose behavior as Checkpoint 1. Stage 2 fine-tuning did not restore structured output behavior.

Failure Case Analysis

The most significant failure pattern was format collapse after Stage 1: Alpaca fine-tuning successfully taught the model to produce helpful, fluent prose but at the cost of spontaneous structured output behavior. This is an important alignment trade-off: general instruction following and strict structured output are not naturally compatible behaviors for a 3.8B model, and fine-tuning on one can suppress the other.

The Stage 2 JSON fine-tuning was insufficiently powerful (500 examples, small model, low LR) to restore structured output discipline. A more effective approach would require either a larger Stage 2 dataset (~2,000–5,000 examples) or a specialized prompting strategy that explicitly conditions the model on output format at inference time.

Discussion: Forgetting vs. Retention

The null forgetting result should be interpreted carefully. The absence of forgetting is largely explained by the extremely small Stage 2 dataset and modest training configuration. In a more realistic scenario, larger dataset, more epochs, higher learning rate, catastrophic forgetting would be a genuine risk.

The literature on sequential fine-tuning suggests that forgetting risk scales with: (1) dissimilarity between Stage 1 and Stage 2 data distributions, (2) the ratio of Stage 2 to Stage 1 dataset size,

and (3) the learning rate used in the later stage. In this experiment, all three factors favored retention: JSON instructions are broadly similar to Alpaca instructions in format, Stage 2 data was 100× smaller, and the learning rate was conservatively set.

When building sequential fine-tuning pipelines, the Stage 2 dataset must be large enough to teach the new capability.

4. Prompt Engineering

Teacher Generation Prompts

The five task-type prompts were designed with the following principles:

1. **Minimal framing:** Each prompt starts with a one-sentence role description, followed by the task instruction, the input, and a JSON trigger to elicit structured output without explanatory prose.
2. **Output constraint enforcement:** Return only valid JSON, no explanation was added after observing that without it, the teacher model frequently produced JSON embedded in a paragraph of explanation.
3. **Iteration:** The `tool_call` prompt was revised after observing a high invalidity rate (~35–40%). The initial prompt produced schema documentation instead of a function call; adding the explicit required fields schema fixed this.
4. **Input diversity:** Seed inputs were designed to cover diverse domains (people, products, medical, financial, geospatial) to prevent the model from learning format through domain-specific patterns.

Judge Evaluation Prompts

The judge prompts were designed to elicit structured verdicts by:

5. Defining all scoring dimensions explicitly with their 1–5 scale meaning.
6. Showing the exact required JSON output format with placeholder values.
7. Placing the two responses after the schema so the model sees the output format before reading the content it must evaluate.

The main limitation observed: Phi-3-mini-128k consistently failed to produce parseable JSON verdicts. Future improvement: A simplified judge prompt asking only for a single-word verdict (“A”, “B”, or “tie”) with a brief reason would dramatically reduce parse failures while still capturing the pairwise preference result.

Appendix: Full Prompt Templates

A. Teacher Generation Prompts

json_extraction.txt

```
You are a precise data extraction assistant. Extract all relevant
entities and attributes from the text below and return them as a
valid JSON object only. No explanation, no markdown, just the JSON.

Text: {{INPUT}}

JSON:
```

schema_constrained.txt

```
You are a structured data generator. Given the schema below, generate
a realistic and valid JSON object that exactly conforms to it.
Return only valid JSON, no explanation.

Schema: {{INPUT}}

JSON:
```

classification.txt

```
You are a text classification assistant. Classify the following text
and return your result as a JSON object with exactly these fields:
{"label": "<category>", "confidence": <0.0-1.0>, "reason": "<one sentence>"}.
Allowed labels: positive, negative, neutral, question, complaint.
Return only valid JSON.

Text: {{INPUT}}

JSON:
```

json_repair.txt

```
You are a JSON repair assistant. The following is malformed JSON.
Fix all syntax errors and return only the corrected valid JSON.
Do not add or remove fields, just fix the syntax.

Malformed JSON: {{INPUT}}

Fixed JSON:
```

tool_call.txt

```
You are a function-calling assistant. Given the user request below,
generate a JSON object with fields: {"tool": "<tool_name>",
"parameters": {<key-value pairs>}}. Return only valid JSON, no explanation.

User request: {{INPUT}}

JSON:
```

B. Judge Evaluation Prompts

alpaca_judge.txt

```
You are an expert judge evaluating two AI responses to an instruction.
Score each response on the following dimensions from 1 (poor) to 5 (excellent):
- instruction_following
- correctness
- clarity
- completeness
- hallucination_risk (5 = no hallucination, 1 = severe)

Instruction: {{INSTRUCTION}}
Input: {{INPUT}}

Response A:
{{RESPONSE_A}}

Response B:
{{RESPONSE_B}}

Return only a JSON object in this exact format:
{
  "response_a_scores": {"instruction_following": 0, "correctness": 0,
    "clarity": 0, "completeness": 0, "hallucination_risk": 0},
  "response_b_scores": {"instruction_following": 0, "correctness": 0,
    "clarity": 0, "completeness": 0, "hallucination_risk": 0},
  "winner": "A|B|tie",
  "justification": "one sentence"
}
```

C. Training Configuration (config.yaml)

```yaml
student_model: "microsoft/Phi-3.5-mini-instruct"
teacher_model: "microsoft/Phi-3-mini-128k-instruct"
judge_model:   "microsoft/Phi-3-mini-128k-instruct"

lora_rank: 16
lora_alpha: 32
lora_dropout: 0.05
lora_target_modules: ["q_proj", "k_proj", "v_proj", "o_proj"]

precision: 4bit
max_seq_length: 1024
batch_size: 4
gradient_accumulation_steps: 4
```

```
stage1_epochs: 3
stage1_lr: 2e-5
stage2_epochs: 3
stage2_lr: 2e-5

num_eval_prompts: 100
eval_max_new_tokens: 512
```

D. References

Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Z., Wang, S., Wang, L., & Chen. W. (2021, October 16). *LoRA: Low-Rank Adaptation of Large Language Models [Paper presentation].* International Conference on Learning Representations (ICLR 2021), Online. https://www.microsoft.com/en-us/research/publication/lora-low-rank-adaptation-of-large-language-models/

Dettmers, T., Pagnoni, A., Holtzman, A & Zettlemoyer, L. (2023, September 21). *QLoRA: Efficient Finetuning of Quantized LLMs [Paper presentation].* 2023 Conference on Neural Information Processing Systems (NeurIPS 2023), New Orleans Ernest N. Morial Convention Center. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html

Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). *Alpaca: A Strong, Replicable Instruction-Following Model.* Stanford Center for Research on Foundation Models. https://crfm.stanford.edu/2023/03/13/alpaca.html

Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D. & Hajishirzi, H. (2023,July). *Self-Instruct: Aligning Language Models with Self-Generated Instructions* [Paper presentation]. Association for Computational Linguistics (ACL 2023), Toronto. https://par.nsf.gov/biblio/10462301-self-instruct-aligning-language-models-self-generated-instructions

Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Lin, Z., Zhang, B., Ni, L., Gao, W., Wang, Y. & Guo, J. (2024). A Survey on LLM-as-a-Judge. *arXiv*. https://arxiv.org/pdf/2411.15594